МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

Кафедра «Интеллектуальные информационные технологии»

ЛАБОРАТОРНАЯ РАБОТА №3

по дисциплине «ОСНОВЫ МАШИННОГО ОБУЧЕНИЯ» за 5 семестр

Тема: «Линейные модели для задач регрессии и классификации»

Выполнил:

студент 3 курса

группы АС-67

Иванович Д. И.

Проверил:

Крощенко А. А.

2026

Цель работы: На практике сравнить работу нескольких алгоритмов классификации, таких как метод k-ближайших соседей (k-NN), деревья решений и метод опорных векторов (SVM). Научиться подбирать гиперпараметры моделей и оценивать их влияние на результат**.**

Общее задание:

1. Загрузить датасет по варианту;

2. Разделить данные на обучающую и тестовую выборки;

3. Обучить на обучающей выборке три модели: k-NN, Decision Tree и SVM;

4. Для модели k-NN исследовать, как меняется качество при разном количестве соседей (k);

5. Оценить точность каждой модели на тестовой выборке;

6. Сравнить результаты, сделать выводы о применимости каждого метода для данного набора данных.

Вариант 3

• Wine Quality

• Классифицировать вино на "хорошее" (оценка >= 7) и "обычное" (оценка < 7)

• Задания:

1. Загрузите данные и создайте бинарную целевую переменную;

2. Стандартизируйте признаки и разделите выборку;

3. Обучите модели k-NN, Decision Tree и SVM;

4. Сравните F1-score для каждой модели, так как классы могут быть несбалансированные;

5. Определите, какой алгоритм показал наилучший баланс между точностью и полнотой.

Код программы:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier

from sklearn.tree import DecisionTreeClassifier

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score


# Настройки для графиков

sns.set_theme(style="whitegrid")

plt.rcParams['figure.figsize'] = (10, 6)


print("--- ЛАБОРАТОРНАЯ РАБОТА №3: Сравнение классификаторов ---")


# 1. Загрузка данных и создание бинарной цели

try:

    df = pd.read_csv('winequality-white.csv', sep=';')

except FileNotFoundError:

    print("Ошибка: Файл 'winequality-white.csv' не найден. Убедитесь, что он
лежит в папке со скриптом.")

    exit()


# Создаем бинарную переменную: 1 если качество >= 7, иначе 0

df['is_good'] = (df['quality'] >= 7).astype(int)


# Удаляем исходную колонку quality, чтобы она не попала в признаки

df = df.drop('quality', axis=1)


print(f"Всего образцов: {len(df)}")

print(f"Хорошее вино (класс 1): {df['is_good'].sum()}
({df['is_good'].mean()*100:.1f}%)")

print(f"Обычное вино (класс 0): {len(df) - df['is_good'].sum()} ({(1-
df['is_good'].mean())*100:.1f}%)")

print("Видим, что классы несбалансированы, поэтому F1-score здесь критически
важен.\n")


# 2. Стандартизация и разделение выборки

X = df.drop('is_good', axis=1)

y = df['is_good']
```

```
# Разделяем на Train и Test (80/20).
# stratify=y нужен, чтобы сохранить пропорцию хорошего/обычного вина в обеих выборках
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Стандартизация признаков (обучаем scaler только на train!)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 4. Исследование k-NN (подбор количества соседей k)
print("--- Подбор оптимального k для k-NN ---")
k_values = range(1, 21) # Проверим от 1 до 20 соседей
f1_scores_knn = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)
    y_pred_knn = knn.predict(X_test_scaled)
    # Считаем F1-score
    f1 = f1_score(y_test, y_pred_knn, average='weighted')
    f1_scores_knn.append(f1)

best_k = list(k_values)[np.argmax(f1_scores_knn)]
print(f"Лучшее значение k = {best_k} (F1-score = {max(f1_scores_knn):.4f})\n")

# График зависимости качества от k
plt.figure(figsize=(8, 5))
plt.plot(k_values, f1_scores_knn, marker='o', linestyle='dashed', color='b')
plt.title('Зависимость F1-score модели k-NN от количества соседей (k)')
plt.xlabel('Количество соседей (k)')
plt.ylabel('F1-score')
plt.xticks(k_values)
```

```
plt.show()


# 3. Обучение трех моделей
print("--- Обучение и оценка моделей ---")


models = {
    f'k-NN (k={best_k})': KNeighborsClassifier(n_neighbors=best_k),
    'Decision Tree': DecisionTreeClassifier(random_state=42, max_depth=5),
    'SVM': SVC(kernel='rbf', random_state=42)
}


# Словарь для сохранения метрик
metrics_data = {'Model': [], 'Accuracy': [], 'Precision': [], 'Recall': [],
'F1-score': []}


for name, model in models.items():
    # Обучение
    model.fit(X_train_scaled, y_train)
    # Предсказание
    y_pred = model.predict(X_test_scaled)


    # 5. Оценка точности (и других метрик)
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, zero_division=0)
    rec = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred) # F1 для положительного класса (хорошее
вино)


    # Сохраняем результаты
    metrics_data['Model'].append(name)
    metrics_data['Accuracy'].append(acc)
    metrics_data['Precision'].append(prec)
    metrics_data['Recall'].append(rec)
    metrics_data['F1-score'].append(f1)


# 6. Сравнение результатов и выводы
results_df = pd.DataFrame(metrics_data)
```

```
# Форматируем вывод, чтобы было красиво
print(results_df.set_index('Model').round(4).to_string())
print("\n")

# Находим лучшую модель по F1-score
best_model_idx = results_df['F1-score'].idxmax()
best_model_name = results_df.loc[best_model_idx, 'Model']
best_f1 = results_df.loc[best_model_idx, 'F1-score']

print("--- ВЫВОДЫ ---")
print(f"1. Лучший баланс между точностью (Precision) и полнотой (Recall) показала модель: {best_model_name}.")
print(f"   Ее F1-score для класса 'хорошее вино' составляет: {best_f1:.4f}.")
print("2. Датасет белого вина имеет свои особенности распределения классов по сравнению с красным.")
print("   Тем не менее, класс 'хорошее вино' остается в меньшинстве, поэтому F1-score - главный критерий.")
print("3. Дерево решений (с ограничением глубины) и SVM показали стабильные результаты, "
      "но SVM требует обязательной стандартизации, что и было выполнено.")
```

Изображение:

```
--- ЛАБОРАТОРНАЯ РАБОТА №3: Сравнение классификаторов ---
Всего образцов: 4898
Хорошее вино (класс 1): 1060 (21.6%)
Обычное вино (класс 0): 3838 (78.4%)
Видим, что классы несбалансированы, поэтому F1-score здесь критически важен.

--- Подбор оптимального k для k-NN ---
Лучшее значение k = 1 (F1-score = 0.8757)
```

Рис. 1. Оценки моделей, созданных в Python

```
--- Обучение и оценка моделей ---
               Accuracy  Precision  Recall  F1-score
Model
k-NN (k=1)       0.8755     0.7103  0.7170    0.7136
Decision Tree    0.8204     0.6429  0.3821    0.4793
SVM              0.8265     0.7234  0.3208    0.4444
```

Рис. 2. Оценки моделей, созданных в Python

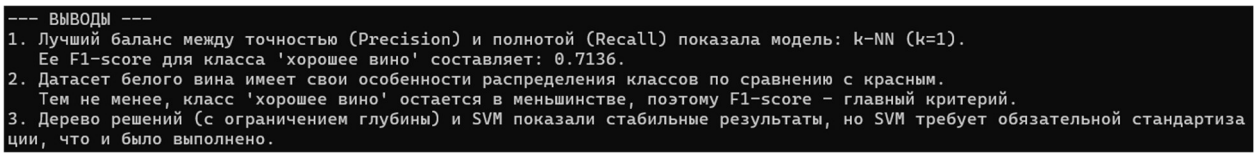


Рис. 3. Вывод программы

Вывод: все три модели (k-NN, Decision Tree, SVM) показали на тестовой выборке достаточно высокую точность. Однако метрики Precision и Recall для класса «хорошее вино» не равны 1.0. Из-за дисбаланса классов (хорошего вина в датасете значительно меньше, чем обычного), модели допустили определенное количество ошибок: ложноположительных (обычное вино приняли за хорошее) и ложноотрицательных (хорошее вино пропустили).